PLAN DE TRABAJO · ÚLTIMA FASE

RSVP, dashboard y Oracle

Invitación de boda Sonia & David · 28 de diciembre de 2026

El modal de confirmación · el correo de agradecimiento · el dashboard del cliente

Autonomous Database Always Free · el driver oracledb · NextAuth.js · Resend

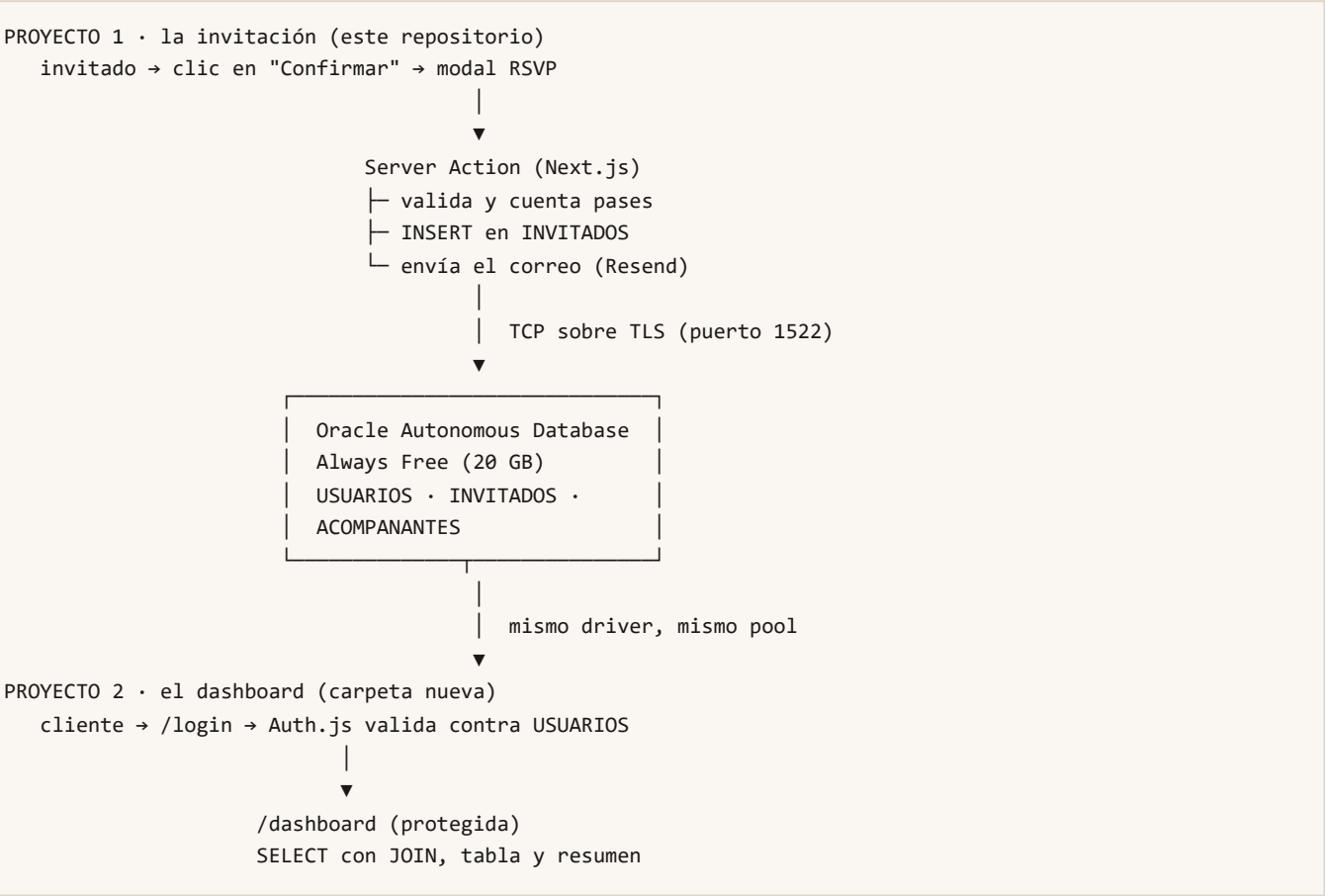
12 de noviembre de 2026

Contenido

1. El panorama: dos proyectos, una base de datos
2. Oracle Autonomous Database
3. Las tablas
4. El archivo de conexión: `lib/oracle.ts`
5. El modal de RSVP en la página de la boda
6. La server action: validar, contar, guardar
7. El correo de confirmación
8. El dashboard del cliente (proyecto nuevo)
9. Variables de entorno y despliegue
10. Seguridad y privacidad
11. Orden de trabajo y checklist

1. El panorama: dos proyectos, una base de datos

Esta fase son **dos aplicaciones** que se conectan a la misma base Oracle. No es un solo proyecto con dos partes: son dos carpetas, dos despliegues y dos dominios distintos.



1.1 Por qué dos proyectos y no uno

Motivo	Detalle
Permisos	El invitado no debe tener acceso al dashboard ni a las credenciales. Separados, no hay forma de equivocarse
Seguridad	La clave del dashboard no vive en el sitio público. Un error en el sitio de la boda no expone el panel
Despliegue	Cada uno se actualiza por separado. Una mejora en el dashboard no obliga a tocar la invitación
Reutilización	Si el desarrollador hace más bodas, el dashboard es el mismo y solo cambia el `USUARIO_ID`

> **Lo único que comparten es la base.** Los dos proyectos instalan `oracledb` y tienen su propio `lib/oracle.ts`. Es el mismo archivo copiado, no un paquete compartido.

1.2 ¿Qué es exactamente "Autonomous Database Always Free"?

Oracle Cloud ofrece bases de datos administradas: Oracle aplica parches, respaldos y ajuste automáticamente. La capa **Always Free** es gratuita sin fecha de expiración e incluye 20 GB. Se maneja desde el navegador (Database Actions), así que no hay servidor que cuidar.

Concepto	Qué significa
Autonomous	El motor se autogestiona: parches, respaldos, índices y rendimiento
Always Free	Nivel gratuito permanente, con 20 GB y límites de CPU/OCPU bajos pero suficientes
ADB	La abreviatura del servicio (Autonomous Database)
Database Actions	Una consola web dentro del mismo servicio para ejecutar SQL sin instalar nada

2. Oracle Autonomous Database

2.1 Crear la instancia (una sola vez)

1. Entra a **oracle.cloud** y crea la cuenta (pide tarjeta, pero el nivel Free no cobra).
2. En **Oracle Cloud Infrastructure**, menú → **Autonomous Database** → **Create**.
3. Tipo: **Transaction Processing**. Nombre y contraseña del usuario `ADMIN` (guárdala en un lugar seguro).
4. Región: la más cercana a México (`mx-queretaro` o `mx-monterrey` si está disponible).
5. Acceso: **Allow secure access from anywhere** (por ahora; se puede restringir después).

2.2 El driver: `oracledb` en modo thin

```
npm install oracledb
```

El driver de Oracle para Node.js tiene dos modos:

Modo	Cómo funciona	Cuándo usarlo
Thin	JavaScript puro. No instala nada del Oracle Client	**Siempre aquí.** Funciona en Vercel sin compilar nativos
Thick	Carga la librería nativa del Oracle Client (Instant Client)	Solo si necesitas funciones avanzadas (Kerberos, PKIX, ...)

> **Thin es la clave del despliegue:** Vercel compila funciones serverless en un entorno donde no puedes instalar librerías nativas. Como thin es JavaScript puro, la conexión funciona tal cual.

2.3 La cadena de conexión

En la consola de la base, **Database connection** → **Connection strings** copia la cadena `TCPS` (puerto 1522). Tiene esta forma:

```
(description=(retry_count=20)(retry_delay=3)
  (address=(protocol=tcps)(port=1522)(host=adb.mx-queretaro-1.oraclecloud.com))
  (connect_data=(service_name=g8f3a2c1b2e_miboda_high.adb.oraclecloud.com))
  (security=(ssl_server_dn_match=yes)))
```

Parte	Qué indica
`protocol=tcps`	TLS cifrado, igual que HTTPS pero en el puerto 1522
`host=adb…oraclecloud.com`	La puerta de entrada de tu base
`service_name=g8f3a2c1b2e_miboda_high`	El "nombre del servicio" dentro de la base. Viene en tres sabores: `_high` , `_medium` , `_low`
`retry_count / retry_delay`	Si la base está ocupada, reintenta 20 veces esperando 3 segundos. Importante en el nivel Free, que se duerme
`ssl_server_dn_match=yes`	Verifica que el certificado del servidor corresponda al servicio

> **Si el driver se queja del certificado** (`DPI-1049` o `SSL handshake failed`), hay dos caminos:
> **A)** En Database Actions → **Database connection** → **Wallet**, descarga el wallet y apunta a la carpeta con `walletLocation` .
> **B)** Para desarrollo, cambia `ssl_server_dn_match=yes` por `no` . Es menos estricto; para producción conviene el wallet.

2.4 El pool de conexiones

Abrir una conexión Oracle cuesta cientos de milisegundos. Un **pool** las abre una vez y las reutiliza. Por eso toda la aplicación comparte *uno* solo.

> **Sin `globalThis` el pool se multiplicaría.** En desarrollo, Next recarga los módulos con cada cambio. Si el pool se creara dentro de la función, cada recarga abriría uno nuevo y llegarías al límite de sesiones del nivel Free.

3. Las tablas

Se crean una sola vez, desde **Database Actions** → **SQL**. Tres tablas: el cliente, sus invitados y (opcional) los acompañantes.

```
--El cliente: quien se loguea al dashboard
CREATE TABLE USUARIOS (
    ID            NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    NOMBRE        VARCHAR2(120) NOT NULL,
    EMAIL         VARCHAR2(160) NOT NULL UNIQUE,
    PASSWORD_HASH VARCHAR2(200) NOT NULL,
    LIMITE_PASES  NUMBER(4)     DEFAULT 100 NOT NULL,
    CREADO_EN     TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL
);

--Cada confirmación de un invitado
CREATE TABLE INVITADOS (
    ID               NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    USUARIO_ID       NUMBER(10)   NOT NULL,
    NOMBRE           VARCHAR2(120) NOT NULL,
    EMAIL            VARCHAR2(160) NOT NULL,
    PASES_PERMITIDOS NUMBER(2)    DEFAULT 1 NOT NULL,
    RESTRICCIONES    VARCHAR2(400),
    CREADO_EN        TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL,
    CONSTRAINT INVITADOS_USUARIO_FK FOREIGN KEY (USUARIO_ID) REFERENCES USUARIOS(ID),
    CONSTRAINT INVITADOS_PASES_CK   CHECK (PASES_PERMITIDOS BETWEEN 1 AND 10)
);

--Opcional: el nombre de cada acompañante
CREATE TABLE ACOMPANANTES (
    ID          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    INVITADO_ID NUMBER(10)   NOT NULL,
    NOMBRE      VARCHAR2(120) NOT NULL,
    CONSTRAINT ACOMPANANTES_INVITADO_FK FOREIGN KEY (INVITADO_ID)
        REFERENCES INVITADOS(ID) ON DELETE CASCADE
);

--Índice para el contador y para el dashboard
CREATE INDEX INVITADOS_USUARIO_IDX ON INVITADOS(USUARIO_ID);
```

Tabla	Para qué	Clave
`USUARIOS`	El cliente del dashboard. `PASSWORD_HASH` guarda el hash de la contraseña, nunca la contraseña	`ID`
`INVITADOS`	Cada confirmación. `PASES_PERMITIDOS` es cuántos pases usó ese invitado (1 si viene solo, más si trae acompañantes)	`ID`, apunta a `USUARIOS`
`ACOMPANANTES`	Solo si quieres el nombre de cada acompañante. Si no, basta con el conteo de `PASES_PERMITIDOS`	apunta a `INVITADOS`

3.1 La contraseña nunca en texto plano

```
npm install bcryptjs
```

```
import bcrypt from "bcryptjs";

//Al crear el usuario
const hash = await bcrypt.hash("miContraseñaSegura", 12);

//Al iniciar sesión
const valida = await bcrypt.compare("laQueEscribieron", hashDeLaBase);
```

El número `12` es el "costo": cuántas vueltas de cifrado. 10–12 es el estándar. Cada vuelta hace el hash más lento de calcular para un atacante, sin afectar apenas al inicio de sesión.

3.2 Consulta de control

```
SELECT
    (SELECT LIMITE_PASES FROM USUARIOS WHERE ID = :usuarioId)           AS LIMITE,
    (SELECT COALESCE(SUM(PASES_PERMITIDOS), 0)
       FROM INVITADOS WHERE USUARIO_ID = :usuarioId)                   AS USADOS
FROM DUAL;
```

`COALESCE` devuelve 0 cuando la suma es `NULL` (o sea, cuando todavía no hay invitados). Sin él, el `NULL` se colaría por todas partes y las comparaciones fallarían.

4. El archivo de conexión: `lib/oracle.ts`

Es **exactamente el mismo archivo** en los dos proyectos. Se guarda en `lib/oracle.ts` (la carpeta `lib` es convención para código de servidor que no es un componente).

```
//lib/oracle.ts
import oracledb from "oracledb";

//En TypeScript, esto le dice al compilador que vamos a colgar
//nuestro pool de la variable global para que sobreviva los recargas de Next
declare global {
    var __oraclePool: oracledb.Pool | undefined;
}

export async function getPool(): Promise<oracledb.Pool> {
    if (!globalThis.__oraclePool) {
        globalThis.__oraclePool = await oracledb.createPool({
            user: process.env.ORACLE_USER,
            password: process.env.ORACLE_PASSWORD,
            connectString: process.env.ORACLE_CONNECT_STRING,

            //Nivel Free: no hace falta más
            poolMin: 1,
            poolMax: 4,
            poolIncrement: 1,
            poolTimeout: 60,
        });
    }

    return globalThis.__oraclePool;
}
```

Parámetro	Por qué
`poolMin: 1`	Mantiene una conexión abierta. En el nivel Free, la primera conexión tarda; tener una lista evita ese salto
`poolMax: 4`	Una boda no necesita más. El nivel Free tiene un límite de sesiones y conviene no acercarse
`poolTimeout: 60`	Si se agotan las conexiones, espera hasta 60 s antes de fallar
`globalThis.__oraclePool`	Sobrevive a los recargas en desarrollo y evita pools duplicados

> **Este archivo nunca se importa desde un componente de cliente.** Si un `"use client"` importa `lib/oracle.ts`, el navegador intentaría descargar `oracledb` y fallaría. Solo lo importan server actions y páginas de servidor.

5. El modal de RSVP en la página de la boda

5.1 Radix UI o Headless UI

Ambas librerías resuelven lo mismo: un modal **accesible** (foco atrapado, `Escape`, `aria-modal`, cierre al hacer clic fuera) sin tener que implementarlo a mano.

Librería	Notas
Radix UI `recomendada`	Sin estilos (headless), primitivas composables, el más usado con React 19. Se instala por piezas: `npm install @radix-ui/react-dialog`
Headless UI	De los creadores de Tailwind. Componentes ya con clases de Tailwind incluidas. Menos flexible para personalizar

> **Dato importante:** ya tienes un modal funcional en el lightbox de la Galería, hecho con el `<dialog>` nativo. El RSVP *podría* usar el mismo enfoque. Pero como pediste una librería, aquí va Radix, que es lo que se usa en producción.

5.2 El componente del modal

```
"use client";

import * as Dialog from "@radix-ui/react-dialog";
import { useActionState } from "react";
import { confirmarAsistencia, type RsvpState } from "@/app/actions/rsvp";

const estadoInicial: RsvpState = { ok: false, message: "" };

type Props = {
    abierto: boolean;
    onCerrar: () => void;
};

export default function RsvpModal({ abierto, onCerrar }: Props) {
    const [estado, formAction, pendiente] = useActionState(confirmarAsistencia, estadoInicial);

    return (
        <Dialog.Root open={abierto} onOpenChange={(open) => { if (!open) onCerrar(); }}>
            <Dialog.Portal>
                <Dialog.Overlay className="fixed inset-0 z-[110] bg-ink/70 backdrop-blur-sm" />

                <Dialog.Content
                    className="fixed left-1/2 top-1/2 z-[120] w-[calc(100%-2rem)] max-w-lg
                               -translate-x-1/2 -translate-y-1/2 rounded-2xl border border-sand/40
                               bg-cream p-6 shadow-2xl">

                    <Dialog.Title className="font-serif text-2xl text-coffee">
                        Confirma tu asistencia
                    </Dialog.Title>

                    <Dialog.Description className="mt-2 font-sans text-sm text-ink/70">
                        Nos encantaría contar contigo.
                    </Dialog.Description>

                    {estado.ok ? (
                        <div className="mt-8 text-center">
                            <p className="font-serif text-2xl text-coffee">¡Gracias por confirmar!</p>
                            <p className="mt-3 font-sans text-sm text-ink/70">{estado.message}</p>

                            <Dialog.Close asChild>
                                <button className="mt-6 rounded-full bg-terracotta px-7 py-3 font-sans
                                                   text-[11px] uppercase tracking-[0.2em] text-white
                                                   transition-colors duration-300 hover:bg-cinnamon">
                                    Cerrar
                                </button>
                            </Dialog.Close>
                        </div>
                    ) : (
                        <form action={formAction} className="mt-6 space-y-5">
                            <div>
                                <label htmlFor="nombre" className="font-sans text-[10px] uppercase
                                                                   tracking-[0.25em] text-ink/70">
                                    Nombre completo
                                </label>
                                <input id="nombre" name="nombre" required minLength={3}
                                       className="mt-2 w-full rounded-xl border border-sand/70 bg-white
                                                  px-4 py-3 font-sans text-base text-ink outline-none
                                                  focus:border-terracotta" />
                            </div>

                            <div>
```

```
                        <label htmlFor="email" className="font-sans text-[10px] uppercase
                                                          tracking-[0.25em] text-ink/70">
                            Correo electrónico
                        </label>
                        <input id="email" name="email" type="email" required
                               className="mt-2 w-full rounded-xl border border-sand/70 bg-white
                                          px-4 py-3 font-sans text-base text-ink outline-none
                                          focus:border-terracotta" />
                    </div>

                    <div>
                        <label htmlFor="asisten" className="font-sans text-[10px] uppercase
                                                           tracking-[0.25em] text-ink/70">
                            ¿Cuántos asisten?
                        </label>
                        <select id="asisten" name="asisten" required defaultValue="1"
                                className="mt-2 w-full rounded-xl border border-sand/70 bg-white
                                           px-4 py-3 font-sans text-base text-ink outline-none
                                           focus:border-terracotta">
                            <option value="1">1 persona</option>
                            <option value="2">2 personas</option>
                            <option value="3">3 personas</option>
                            <option value="4">4 personas</option>
                        </select>
                    </div>

                    <div>
                        <label htmlFor="restricciones" className="font-sans text-[10px] uppercase
                                                                 tracking-[0.25em] text-ink/70">
                            Restricciones alimenticias
                        </label>
                        <textarea id="restricciones" name="restricciones" rows={3}
                                  className="mt-2 w-full rounded-xl border border-sand/70 bg-white
                                             px-4 py-3 font-sans text-base text-ink outline-none
                                             focus:border-terracotta" />
                    </div>

                    {estado.message ? (
                        <p role="alert" className="font-sans text-sm text-red-700">
                            {estado.message}
                        </p>
                    ) : null}

                    <button type="submit" disabled={pendiente}
                            className="w-full rounded-full bg-terracotta px-6 py-3 font-sans
                                       text-[11px] uppercase tracking-[0.2em] text-white
                                       transition-colors duration-300 hover:bg-cinnamon
                                       disabled:opacity-60">
                        {pendiente ? "Enviando..." : "Confirmar asistencia"}
                    </button>
                </form>
            )}

            <Dialog.Close asChild>
                <button aria-label="Cerrar"
                        className="absolute right-4 top-4 font-sans text-lg text-ink/50
                                   transition-colors duration-200 hover:text-ink">
                    ×
                </button>
            </Dialog.Close>
        </Dialog.Content>
    </Dialog.Portal>
</Dialog.Root>
```

```
    );
}
```

5.3 Las piezas de Radix

Pieza	Qué hace
`Dialog.Root`	Dueño del estado abierto/cerrado. Recibe `open` y `onOpenChange`
`Dialog.Portal`	Mueve el contenido al `<body>`. Sin esto, un `overflow: hidden` de un padre lo recortaría
`Dialog.Overlay`	El fondo oscuro. Recibe los clics y cierra
`Dialog.Content`	La caja del modal. Aquí va todo el formulario
`Dialog.Title` / `Description`	Textos que el lector de pantalla anuncia. **Obligatorios** para accesibilidad
`Dialog.Close asChild`	Convierte lo que sea en el botón de cerrar. `asChild` fusiona las props con el hijo

5.4 Abrirlo desde el Header

El Header ya es un componente de cliente, así que puede tener el estado del modal. Cambia el enlace "Confirmar" por un botón y monta el modal al final del `<header>`:

```
//components/layout/Header.tsx
const [rsvpAbierto, setRsvpAbierto] = useState(false);

//El botón de escritorio
<button
    type="button"
    onClick={() => setRsvpAbierto(true)}
    className="rounded-full bg-terracotta px-5 py-2.5 font-sans text-[11px] uppercase
               tracking-[0.2em] text-white transition-colors duration-300 hover:bg-cinnamon">
    Confirmar
</button>

//El botón del menú móvil: lo mismo, con onClick={() => { setIsMenuOpen(false); setRsvpAbierto(true); }}

//Al final del header, antes de cerrar
<RsvpModal abierto={rsvpAbierto} onCerrar={() => setRsvpAbierto(false)} />
```

> **Por qué el estado vive en el Header y no en la página:** el botón que abre el modal está ahí. Si el estado viviera en `page.tsx` (un componente de servidor), haría falta un contexto o pasar props a través del árbol.

6. La server action: validar, contar, guardar

Una **server action** es una función que corre en el servidor pero se llama desde el navegador como si fuera local. Next le pone el transporte, la serialización y la protección contra CSRF.

```
//app/actions/rsvp.ts
"use server";

import oracledb from "oracledb";
import { getPool } from "@/lib/oracle";
import { enviarConfirmacion } from "@/lib/email";

export type RsvpState = {
    ok: boolean;
    message: string;
};

export async function confirmarAsistencia(
    _prev: RsvpState,
    formData: FormData,
): Promise<RsvpState> {
    //1. Leer y limpiar
    const nombre = String(formData.get("nombre") ?? "").trim();
    const email = String(formData.get("email") ?? "").trim().toLowerCase();
    const asisten = Number(formData.get("asisten"));
    const restricciones = String(formData.get("restricciones") ?? "").trim();

    //2. Validar (la del navegador es cortesía; esta es la que vale)
    if (nombre.length < 3) {
        return { ok: false, message: "Escribe tu nombre completo." };
    }
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
        return { ok: false, message: "Escribe un correo electrónico válido." };
    }
    if (!Number.isInteger(asisten) || asisten < 1 || asisten > 10) {
        return { ok: false, message: "El número de asistentes no es válido." };
    }

    const usuarioId = Number(process.env.NEXT_PUBLIC_USUARIO_ID ?? 1);

    try {
        const pool = await getPool();

        //3. El contador: cuántos pases ya están comprometidos
        const usadosResult = await pool.execute(
            `SELECT COALESCE(SUM(PASES_PERMITIDOS), 0) AS USADOS
             FROM INVITADOS WHERE USUARIO_ID = :usuarioId`,
            { usuarioId },
        );
        const usados = Number((usadosResult.rows?.[0] as unknown[])?.[0] ?? 0);

        //4. El límite del cliente
        const limiteResult = await pool.execute(
            `SELECT LIMITE_PASES FROM USUARIOS WHERE ID = :usuarioId`,
            { usuarioId },
        );
        const limite = Number((limiteResult.rows?.[0] as unknown[])?.[0] ?? 0);

        if (usados + asisten > limite) {
            return {
                ok: false,
                message: `Lo sentimos: sólo quedan ${limite - usados} ${limite - usados === 1 ? "pase" :
"pases"}.`,
            };
        }

        //5. Guardar
        await pool.execute(
```

```
            `INSERT INTO INVITADOS (USUARIO_ID, NOMBRE, EMAIL, PASES_PERMITIDOS, RESTRICCIONES)
             VALUES (:usuarioId, :nombre, :email, :asisten, :restricciones)`,
            { usuarioId, nombre, email, asisten, restricciones },
            { autoCommit: true },
        );

        //6. Avisar al invitado (sin tumbar la petición si el correo falla)
        try {
            await enviarConfirmacion({ para: email, nombre, asisten });
        } catch (error) {
            console.error("No se pudo enviar el correo de confirmación", error);
        }

        return { ok: true, message: "Te llegará un correo con los detalles. ¡Nos vemos ahí!" };
    } catch (error) {
        console.error("Error al guardar el RSVP", error);
        return { ok: false, message: "No pudimos guardar tu confirmación. Intenta de nuevo." };
    }
}
```

6.1 El contador y el límite, paso a paso

```
1. invitados ya confirmados
   SELECT COALESCE(SUM(PASES_PERMITIDOS), 0)  →  USADOS = 47

2. tope del cliente
   SELECT LIMITE_PASES FROM USUARIOS        →  LIMITE = 100

3. comparar
   USADOS + asisten  >  LIMITE ?
   47 + 5 = 52  <= 100   →  se guarda
   47 + 60 = 107 > 100   →  se rechaza con un mensaje
```

> **Esto no es a prueba de concurrencia.** Si dos invitados confirman al mismo tiempo y ambos ven 95/100, los dos pasarían y quedarían 105. Para una boda es aceptable. La forma rigurosa es un `SELECT ... FOR UPDATE` sobre la fila del usuario o una restricción única. Si algún día importa, se implementa.

6.2 Bind variables: la parte de seguridad

Nunca se concatena texto del usuario dentro del SQL. Se usan **bind variables**: el SQL lleva `:nombre` y los valores viajan aparte.

```
//❌ Mal: inyección SQL. Con nombre = "x'); DROP TABLE INVITADOS; --" se borra la tabla
await pool.execute(`INSERT INTO INVITADOS (NOMBRE) VALUES ('${nombre}')`);

//✅ Bien: el valor nunca se interpreta como SQL
await pool.execute("INSERT INTO INVITADOS (NOMBRE) VALUES (:nombre)", { nombre });
```

Además de la seguridad, los binds permiten que Oracle reutilice el plan de ejecución: es más rápido.

6.3 `useActionState`: el puente con el formulario

Devuelve	Para qué
`estado`	El resultado: `{ ok, message }`
`formAction`	Se pasa al `action` del `<form>`
`pendiente`	`true` mientras se ejecuta. Sirve para deshabilitar el botón y evitar dobles envíos

El primer argumento de la acción (`_prev`) es el estado anterior. Next lo rellena solo; por eso el nombre empieza con guion bajo (se recibe pero no se usa).

7. El correo de confirmación

El correo lo envía **el servidor**, nunca el navegador: si se hiciera desde el cliente, la clave de la API quedaría pública.

7.1 Resend (recomendado)

```
npm install resend
```

```
//lib/email.ts
import { Resend } from "resend";

const resend = new Resend(process.env.RESEND_API_KEY);

export async function enviarConfirmacion({
    para,
    nombre,
    asisten,
}: {
    para: string;
    nombre: string;
    asisten: number;
}) {
    await resend.emails.send({
        from: "Boda Sonia y David <no-reply@tudominio.com>",
        to: [para],
        subject: "¡Tu asistencia está confirmada!",
        html: `
            <div style="font-family: Georgia, serif; color: #1a1817;">
                <h1 style="color: #5c3a21;">¡Gracias, ${nombre}!</h1>
                <p style="font-family: sans-serif; font-size: 15px;">
                    Recibimos tu confirmación para <strong>${asisten}
                    ${asisten === 1 ? "persona" : "personas"}</strong>.
                </p>
                <p style="font-family: sans-serif; font-size: 15px;">
                    Nos vemos el 28 de diciembre de 2026 en el Centro Cristiano Izcalli.
                </p>
            </div>`,
    });
}
```

> **El remitente debe ser un dominio verificado.** Resend exige verificar `tudominio.com` (agregando registros DNS) antes de enviar. No puedes usar un correo Gmail. Si no tienes dominio, la alternativa es Nodemailer.

7.2 Nodemailer (alternativa con Gmail)

```
npm install nodemailer
```

```
//lib/email.ts
import nodemailer from "nodemailer";

const transporte = nodemailer.createTransport({
    host: process.env.SMTP_HOST,           // p. ej. "smtp.gmail.com"
    port: Number(process.env.SMTP_PORT ?? 587),
    secure: false,                          // 587 usa STARTTLS
    auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS,       // contraseña de aplicación, no la normal
    },
});

export async function enviarConfirmacion({ para, nombre, asisten }) {
    await transporte.sendMail({
        from: `"Boda Sonia y David" <${process.env.SMTP_USER}>`,
        to: para,
        subject: "¡Tu asistencia está confirmada!",
        html: `<p>Gracias ${nombre}, recibimos ${asisten} ${asisten === 1 ? "confirmación" :
"confirmaciones"}.</p>`,
    });
}
```

Con Gmail hay que crear una **contraseña de aplicación** (Google → Seguridad → Verificación en dos pasos → Contraseñas de aplicación). La contraseña normal no funciona.

7.3 Qué pasa si el correo falla

En la server action, el envío está envuelto en su propio `try` **después** del INSERT. Así, si Resend o Nodemailer falla, la confirmación ya está guardada y el invitado ve el "¡Gracias!" de todos modos. El fallo se registra en la consola del servidor.

> **Orden correcto:** guardar primero, enviar después. Al revés, un fallo de correo dejaría al invitado sin confirmación a pesar de haberla hecho.

8. El dashboard del cliente (proyecto nuevo)

8.1 Crear el proyecto

```
npx create-next-app@latest dashboard
#TypeScript, Tailwind, App Router, alias @/*
```

Copia `lib/oracle.ts` tal cual. La paleta de colores es la misma, definida en `app/globals.css` con los mismos tokens:

```
@theme {
  --color-cream: #f3ebe1;
  --color-sand: #d8bfa8;
  --color-terracotta: #c08d52;
  --color-cinnamon: #a66443;
  --color-coffee: #5c3a21;
  --color-ink: #1a1817;
}
```

8.2 NextAuth.js (Auth.js) v5

```
npm install next-auth@beta bcryptjs
```

Se configura en un archivo `auth.ts` en la raíz del proyecto. Como el login es con correo y contraseña, se usa el **proveedor de credenciales**.

```
//auth.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";
import bcrypt from "bcryptjs";
import { getPool } from "@/lib/oracle";

export const { handlers, auth, signIn, signOut } = NextAuth({
    //Necesario en Vercel: Auth.js tiene que saber que el host es de confianza
    trustHost: true,

    //Con credenciales no hay flujo OAuth, así que la sesión es un JWT
    session: { strategy: "jwt" },

    providers: [
        Credentials({
            credentials: {
                email: { label: "Correo", type: "email" },
                password: { label: "Contraseña", type: "password" },
            },

            async authorize(credentials) {
                const email = String(credentials?.email ?? "").trim().toLowerCase();
                const password = String(credentials?.password ?? "");

                if (!email || !password) return null;

                const pool = await getPool();
                const resultado = await pool.execute(
                    `SELECT ID, NOMBRE, EMAIL, PASSWORD_HASH, LIMITE_PASES
                     FROM USUARIOS WHERE LOWER(EMAIL) = :email`,
                    { email },
                );

                const fila = resultado.rows?.[0] as unknown[] | undefined;
                if (!fila) return null;

                const [id, nombre, correo, passwordHash, limitePases] = fila;
                const valida = await bcrypt.compare(password, String(passwordHash));

                if (!valida) return null;

                //Esto es lo que Auth.js guarda en el token
                return {
                    id: String(id),
                    name: String(nombre),
                    email: String(correo),
                    limitePases: Number(limitePases),
                };
            },
        }),
    ],

    callbacks: {
        //Copia datos del usuario al token JWT
        async jwt({ token, user }) {
            if (user) {
                token.uid = String(user.id);
                token.limitePases = (user as { limitePases?: number }).limitePases ?? 0;
            }
            return token;
        },
        //Y del token a la sesión que ve la página
        async session({ session, token }) {
```

```
            if (session.user) {
                (session.user as unknown as { id: string }).id = token.uid as string;
                (session.user as unknown as { limitePases: number }).limitePases =
                    token.limitePases as number;
            }
            return session;
        },
    },
});
```

8.3 El endpoint de Auth.js

```
//app/api/auth/[...nextauth]/route.ts
import { handlers } from "@/auth";

export const { GET, POST } = handlers;
```

Los corchetes en el nombre de carpeta son un **segmento dinámico**: Next crea una ruta que responde a `/api/auth/*`, que es lo que Auth.js necesita para el login, el logout y la sesión.

8.4 La página de login

```
"use client";

import { signIn } from "next-auth/react";
import { useState } from "react";

export default function LoginPage() {
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [error, setError] = useState(null);
    const [cargando, setCargando] = useState(false);

    const onSubmit = async (evento: React.FormEvent) => {
        evento.preventDefault();
        setCargando(true);
        setError(null);

        const respuesta = await signIn("credentials", {
            email,
            password,
            redirect: false,
        });

        if (respuesta?.error) {
            setError("Correo o contraseña incorrectos.");
            setCargando(false);
            return;
        }

        window.location.href = "/dashboard";
    };

    return (
        <main className="flex min-h-svh items-center justify-center bg-cream px-6">
            <form onSubmit={onSubmit} className="w-full max-w-sm rounded-2xl
                                        border border-sand/60 bg-white p-8">
                <h1 className="font-serif text-3xl text-coffee">Panel del cliente</h1>

                <label htmlFor="email" className="mt-6 block font-sans text-[10px]
                                        uppercase tracking-[0.25em] text-ink/70">
                    Correo
                </label>
                <input id="email" type="email" value={email}
                       onChange={(e) => setEmail(e.target.value)}
                       className="mt-2 w-full rounded-xl border border-sand/70 px-4 py-3
                                  font-sans text-base text-ink outline-none focus:border-terracotta" />

                <label htmlFor="password" className="mt-4 block font-sans text-[10px]
                                          uppercase tracking-[0.25em] text-ink/70">
                    Contraseña
                </label>
                <input id="password" type="password" value={password}
                       onChange={(e) => setPassword(e.target.value)}
                       className="mt-2 w-full rounded-xl border border-sand/70 px-4 py-3
                                  font-sans text-base text-ink outline-none focus:border-terracotta" />

                {error ? <p role="alert" className="mt-4 font-sans text-sm text-red-700">{error}</p> : null}

                <button type="submit" disabled={cargando}
                        className="mt-6 w-full rounded-full bg-terracotta px-6 py-3 font-sans
                                   text-[11px] uppercase tracking-[0.2em] text-white
                                   transition-colors duration-300 hover:bg-cinnamon disabled:opacity-60">
```

```
                    {cargando ? "Entrando..." : "Entrar"}
                </button>
            </form>
        </main>
    );
}
```

8.5 Proteger el dashboard (sin middleware)

El middleware de Next corre en el **Edge Runtime**, un entorno limitado donde `oracledb` puede fallar. Para un dashboard, la forma más simple y segura es preguntar por la sesión **dentro de la página de servidor**:

```
//app/dashboard/page.tsx
import { auth } from "@/auth";
import { signOut } from "@/auth";
import { redirect } from "next/navigation";
import { getPool } from "@/lib/oracle";

export default async function DashboardPage() {
    const session = await auth();

    //Si no hay sesión, afuera
    if (!session?.user) {
        redirect("/login");
    }

    const usuarioId = (session.user as unknown as { id: string }).id;
    const pool = await getPool();

    //Los invitados, con el nombre de cada acompañante (si se guardó)
    const invitados = await pool.execute(
        `SELECT i.ID, i.NOMBRE, i.EMAIL, i.PASES_PERMITIDOS, i.RESTRICCIONES, i.CREADO_EN
         FROM INVITADOS i
         WHERE i.USUARIO_ID = :usuarioId
         ORDER BY i.CREADO_EN DESC`,
        { usuarioId },
    );

    //El resumen
    const resumen = await pool.execute(
        `SELECT COALESCE(SUM(PASES_PERMITIDOS), 0) AS USADOS,
                COUNT(*) AS CONFIRMACIONES
         FROM INVITADOS WHERE USUARIO_ID = :usuarioId`,
        { usuarioId },
    );

    const fila = resumen.rows?.[0] as unknown[] | undefined;
    const usados = Number(fila?.[0] ?? 0);
    const confirmaciones = Number(fila?.[1] ?? 0);
    const limite = (session.user as unknown as { limitePases: number }).limitePases;
    const disponibles = Math.max(limite - usados, 0);

    return (
        <main className="min-h-svh bg-cream px-6 py-12">
            <div className="mx-auto max-w-5xl">
                <div className="flex items-center justify-between gap-4">
                    <h1 className="font-serif text-3xl text-coffee">Invitados</h1>
                    <form action={async () => { "use server"; await signOut({ redirectTo: "/login" }); }}>
                        <button className="rounded-full border border-sand px-5 py-2 font-sans
                                           text-[11px] uppercase tracking-[0.2em] text-coffee
                                           transition-colors duration-300 hover:bg-sand/20">
                            Cerrar sesión
                        </button>
                    </form>
                </div>

                {/*Tarjetas de resumen */}
                <div className="mt-8 grid gap-4 sm:grid-cols-3">
                    <div className="rounded-2xl border border-sand/60 bg-white p-6">
                        <p className="font-sans text-[10px] uppercase tracking-[0.25em] text-terracotta">
                            Confirmaciones
                        </p>
                        <p className="mt-2 font-serif text-3xl text-coffee">{confirmaciones}</p>
                    </div>
                    <div className="rounded-2xl border border-sand/60 bg-white p-6">
```

```
                        <p className="font-sans text-[10px] uppercase tracking-[0.25em] text-terracotta">
                            Pases usados
                        </p>
                        <p className="mt-2 font-serif text-3xl text-coffee">{usados} / {limite}</p>
                    </div>
                    <div className="rounded-2xl border border-sand/60 bg-white p-6">
                        <p className="font-sans text-[10px] uppercase tracking-[0.25em] text-terracotta">
                            Pases disponibles
                        </p>
                        <p className="mt-2 font-serif text-3xl text-coffee">{disponibles}</p>
                    </div>
                </div>

                {/*La tabla */}
                <div className="mt-8 overflow-hidden rounded-2xl border border-sand/60 bg-white">
                    <table className="w-full font-sans text-sm text-ink">
                        <thead className="bg-sand/20 text-left">
                            <tr>
                                <th className="px-4 py-3">Nombre</th>
                                <th className="px-4 py-3">Correo</th>
                                <th className="px-4 py-3 text-center">Asisten</th>
                                <th className="px-4 py-3">Restricciones</th>
                            </tr>
                        </thead>
                        <tbody>
                            {(invitados.rows ?? []).map((fila, indice) => {
                                const [id, nombre, email, asisten, restricciones] = fila as unknown[];
                                return (
                                    <tr key={String(id)} className={indice % 2 === 0 ? "bg-white" : "bg-
cream/50"}>
                                        <td className="px-4 py-3">{String(nombre)}</td>
                                        <td className="px-4 py-3">{String(email)}</td>
                                        <td className="px-4 py-3 text-center">{Number(asisten)}</td>
                                        <td className="px-4 py-3">{String(restricciones ?? "—")}</td>
                                    </tr>
                                );
                            })}
                        </tbody>
                    </table>
                </div>
            </div>
        </main>
    );
}
```

Sobre TanStack Table y shadcn/ui: para esta tabla no hacen falta. Una tabla de HTML con Tailwind resuelve la necesidad. TanStack Table sirve cuando hay ordenamiento, filtros o paginación con miles de filas; shadcn/ui es útil si quieres componentes ya montados (botones, diálogos, paginación). Se pueden agregar después sin rehacer la página.

9. Variables de entorno y despliegue

9.1 El sitio de la boda

```
#.env.local  (nunca se sube a Git)
ORACLE_USER=ADMIN
ORACLE_PASSWORD=tu_contraseña
ORACLE_CONNECT_STRING=(description=(retry_count=20)(retry_delay=3)(address=(protocol=tcps)(port=1522)
(host=adb.mx-queretaro-1.oraclecloud.com))(connect_data=
(service_name=g8f3a2c1b2e_miboda_high.adb.oraclecloud.com))(security=(ssl_server_dn_match=yes)))

NEXT_PUBLIC_USUARIO_ID=1
RESEND_API_KEY=re_tu_clave
```

9.2 El dashboard

```
#.env.local del proyecto nuevo
ORACLE_USER=ADMIN
ORACLE_PASSWORD=tu_contraseña
ORACLE_CONNECT_STRING=(...la misma...)

#Auth.js: un texto largo y aleatorio, sólo para este proyecto
AUTH_SECRET=un_texto_largo_y_aleatorio
AUTH_URL=https://dashboard-tu-dominio.vercel.app
```

Variable	¿Pública?	Por qué
`ORACLE_USER` / `ORACLE_PASSWORD`	`nunca`	Si llevan `NEXT_PUBLIC_`, se incrustan en el JavaScript del navegador y cualquiera los ve con F12
`ORACLE_CONNECT_STRING`	`nunca`	Contiene el host y el servicio; no es secreto, pero tampoco debe exponerse
`NEXT_PUBLIC_USUARIO_ID`	`pública`	Es sólo el número de cliente. No es secreto y la server action la lee en el servidor igual
`RESEND_API_KEY` / `SMTP_*`	`nunca`	Una clave de envío permite mandar correos a nombre del cliente
`AUTH_SECRET`	`nunca`	Firma las cookies de sesión. Si se filtra, cualquiera puede falsificar una sesión

> **Genera `AUTH_SECRET` con:** `openssl rand -base64 32` en la terminal. Guárdala y no la compartas.

9.3 Despliegue

1. **Sitio de la boda:** Vercel → importa el repositorio → agrega las variables de entorno del apartado 9.1 → Deploy.
2. **Dashboard:** otro proyecto en Vercel con su propio repositorio y sus variables.
3. **Oracle:** en Database Actions → **Access Control List**, permite `0.0.0.0/0` (Vercel no tiene IP fija) o, mejor, las IPs del rango de Vercel.
4. **Prueba:** envía una confirmación y verifica con `SELECT * FROM INVITADOS`.

> **Permitir `0.0.0.0/0` significa que cualquiera puede intentar conectarse a tu base.** La contraseña del usuario `ADMIN` es la única defensa. Usa una larga y aleatoria, y rota la contraseña si alguna vez la escribes en un lugar inseguro.

10. Seguridad y privacidad

Riesgo	Medida
Inyección SQL	Bind variables en **todas** las consultas. Nunca concatenar lo que escribió el usuario
Contraseñas filtradas	Hash con bcrypt. Nunca texto plano, nunca reversible
Sesiones falsas	`AUTH_SECRET` larga, cookies httpOnly (Auth.js las pone así por defecto)
Claves en el navegador	Variables sin `NEXT_PUBLIC_` en todo lo sensible
Spam en el RSVP	Límite de pases, un `input` trampa oculto (honeypot) y, si hace falta, un límite por IP
Datos personales	Pide sólo lo necesario. El invitado debe poder pedir que borres sus datos: un `DELETE` por correo es suficiente para una boda
Correos registrados	No escribas el contenido del correo en `console.log`

10.1 El campo trampa (honeypot)

Un input invisible para los humanos pero visible para los bots. Si se llena, se descarta la confirmación en silencio.

```
<input type="text" name="empresa" tabIndex={-1} autoComplete="off"
       className="hidden" aria-hidden="true" />
```

```
//En la server action
if (String(formData.get("empresa") ?? "").length > 0) {
    //Parece un bot: fingimos éxito para no dar pistas
    return { ok: true, message: "¡Gracias por confirmar!" };
}
```

11. Orden de trabajo y checklist

11.1 En qué orden hacerlo

Paso	Qué	Dónde se verifica
1	Crear la instancia Oracle y ejecutar el DDL	Database Actions → SQL → `SELECT * FROM USUARIOS`
2	Agregar `lib/oracle.ts` y probar la conexión con un script	Consola local: `node --env-file=.env.local scripts/probar-oracle.mjs`
3	Crear el usuario `ADMIN` del dashboard con bcrypt	`SELECT ID, EMAIL FROM USUARIOS`
4	Sitio: instalar Radix, crear el modal y conectarlo al Header	El modal se abre y se cierra con teclado
5	Sitio: la server action (validar, contar, insertar)	Una confirmación de prueba y un `SELECT`
6	Sitio: el correo con Resend o Nodemailer	Recibes el correo en tu bandeja
7	Dashboard: proyecto nuevo, Auth.js, login, dashboard, logout	Entras y ves la fila de prueba
8	Desplegar los dos y probar de nuevo desde la URL pública	Confirmación real + fila visible

11.2 Script de prueba de la conexión

```
//scripts/probar-oracle.mjs
import oracledb from "oracledb";

const conexion = await oracledb.getConnection({
    user: process.env.ORACLE_USER,
    password: process.env.ORACLE_PASSWORD,
    connectString: process.env.ORACLE_CONNECT_STRING,
});

const resultado = await conexion.execute("SELECT SYSTIMESTAMP FROM dual");
console.log("Conectado:", resultado.rows[0][0]);

await conexion.close();
```

```
node --env-file=.env.local scripts/probar-oracle.mjs
```

> **Por qué probar antes de escribir la aplicación:** si la cadena de conexión está mal, el error es del todo o nada. Confirmar que la base responde con `SYSTIMESTAMP` te ahorra horas de depurar código que en realidad está bien.

11.3 Checklist final

- El modal abre desde los dos botones del Header (escritorio y móvil).
- Sin sesión no se puede entrar al dashboard.
- La confirmación guardada aparece en la tabla en menos de un segundo.
- El correo llega y dice el número correcto de asistentes.

- Al llegar al límite, el mensaje es claro y no se guarda nada.
- Ninguna credencial aparece en el JavaScript del navegador.
- `npx tsc --noEmit` y `npx eslint .` limpios en los dos proyectos.

> **Regla para no perderse:** primero la base y la conexión, después el formulario, después el correo, y al final el dashboard. Cada pieza se puede probar por separado; si algo falla, el problema está en lo último que agregaste.